Estructuras de Datos y Algoritmos

Tema 13: Heaps

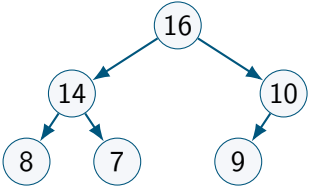
Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Heap binario

Definición: heap binario

Una estructura representada mediante un arreglo que podemos interpretar como un árbol binario completo y cuyos valores satisfacen la propiedad de max-heap o de min-heap.



Todos los niveles anteriores al último están llenos; el último se ocupa de **de izquierda a derecha**.

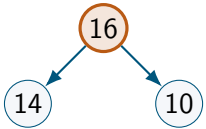
El heap vacío no tiene raíz.

Orden del heap

Definición: max-heap

Para cada hijo i:

$$A[\text{Padre}(i)] \geq A[i].$$

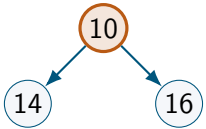


El máximo está en la raíz.

Definición: min-heap

Para cada hijo i:

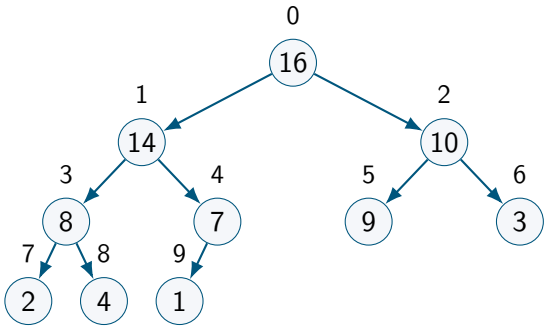
$$A[\text{Padre}(i)] \leq A[i].$$



El mínimo está en la raíz.

Se permiten valores repetidos. No se exige un orden entre hermanos. Un heap no es necesariamente un ABB.

Árbol y arreglo



La raíz ocupa el índice **0**.

El recorrido por niveles determina la posición de cada elemento en el arreglo.

Las relaciones padre–hijo se calculan mediante índices; no almacenamos referencias a los hijos ni al padre.

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Índices

Usamos la representación con raíz en 0, como en la clase de árboles.

Función Padre(i), con $i > 0$

Retornar $\lfloor (i-1)/2 \rfloor$

Función Izquierdo(i)

Retornar $2i + 1$

Función Derecho(i)

Retornar $2i + 2$

```
def padre(i):
    return (i - 1) // 2

def izquierdo(i):
    return 2 * i + 1

def derecho(i):
    return 2 * i + 2
```

Ejemplo: Padre(4) = 1, Izquierdo(1) = 3, Derecho(1) = 4.

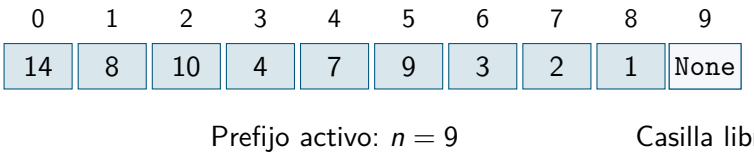
Calcular un índice cuesta $\Theta(1)$. Después comprobamos si el hijo existe.

Tamaño activo

Tamaño y capacidad

n = tamaño es el número de elementos que pertenecen al heap; m = len(datos) es la capacidad del arreglo.

$$0 \leq n \leq m, \qquad \text{índices activos: } 0, \ldots, n-1.$$



El heap tiene 9 elementos y queda una casilla libre, marcada con `None`.

Al insertar un nuevo elemento, n pasa de 9 a 10, mientras que la capacidad m permanece en 10.

Altura y tamaño

Para cualquier árbol binario con $n \geq 1$ nodos $\lceil \log_2(n+1) \rceil - 1 \leq h \leq n-1$.
Un heap de altura h llena los niveles $1, \ldots, h$ y tiene al menos un nodo en el nivel $h+1$.
Usando la misma suma por niveles:

$$n \geq \sum_{i=1}^{h} 2^{i-1} + 1 = \frac{2^h - 1}{2 - 1} + 1 = 2^h.$$

Tomando logaritmos:

$$2^h \leq n \quad \Longrightarrow \quad h \leq \log_2 n < \log_2(n+1).$$

Como h es entero, $h \leq \lceil \log_2(n+1) \rceil - 1$. Las cotas coinciden.

Resultado: altura de un heap

$$h = \lceil \log_2(n+1) \rceil - 1.$$

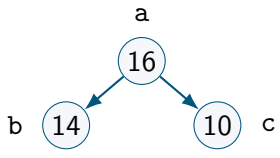
Representación en Python

Cada nodo guarda su clave en `dato` y su posición en `indice`.

```
class NodoHeap:
    def __init__(self, dato):
        self.dato = dato
        self.indice = None
```

```
class HeapMaximo:
    def __init__(self, datos, tamano):
        self.datos = datos
        self.tamano = tamano
        for i in range(tamano):
            datos[i].indice = i
```

```
a = NodoHeap(16)
b = NodoHeap(14)
c = NodoHeap(10)
datos = [a, b, c]
H = HeapMaximo(datos, len(datos))
```



Raíz: a. Ya es un max-heap.

Intercambiar

Método Intercambiar(i, j)

Intercambiar datos[i] y datos[j]
datos[i].indice $\leftarrow i$
datos[j].indice $\leftarrow j$

```
def intercambiar(self, i, j):
    A = self.datos
    A[i], A[j] = A[j], A[i]
    A[i].indice = i
    A[j].indice = j
```

Se intercambian referencias a objetos, sin copiar sus datos.

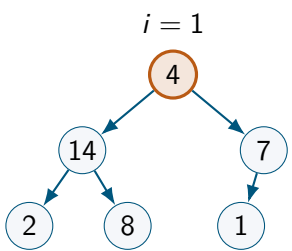
Ejemplo: si un objeto pasa de la casilla 8 a la 3, su `indice` debe cambiar de 8 a 3.

Costo: $\Theta(1)$. Consultar y actualizar esta posición no añade una búsqueda lineal.

Hundir un nodo

Precondición de Hundir(i)

Los subárboles de los hijos de i ya son max-heaps. Solo la clave de i puede ser menor que la de alguno de sus hijos.



Elegimos el mayor entre el elemento y sus hijos existentes.

Si gana un hijo, intercambiamos y continuamos en ese subárbol.

Una llamada en la raíz no construye por sí sola un heap a partir de cualquier arreglo.

Hundir

Método de `HeapMaximo`; n = tamano.

Método Hundir(i)

$l \leftarrow$ Izquierdo(i)
$r \leftarrow$ Derecho(i)
mayor $\leftarrow i$
Si $l < n$ y datos[l].dato $>$ datos[i].dato:
  mayor $\leftarrow l$
Si $r < n$ y datos[r].dato $>$ datos[mayor].dato:
  mayor $\leftarrow r$
Si mayor $\neq i$:
  Intercambiar(i, mayor)
  Hundir(mayor)

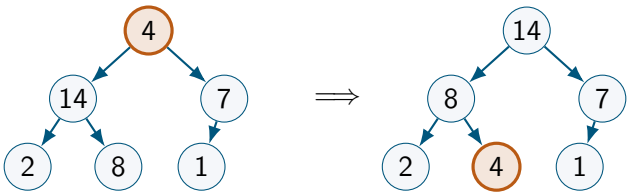
```
def hundir(self, i):
    A, n = self.datos, self.tamano
    l, r = izquierdo(i), derecho(i)
    mayor = i
    if l < n and A[l].dato > A[i].dato:
        mayor = l
    if (r < n and
            A[r].dato > A[mayor].dato):
        mayor = r
    if mayor != i:
        self.intercambiar(i, mayor)
        self.hundir(mayor)
```

Costo: peor caso $O(\log_2(n+1))$; si ya cumple la propiedad, $\Theta(1)$.

Seguimiento

Aplicamos `hundir(0)` al arreglo de seis claves.

Paso	Claves en el arreglo
Inicial	**4**, 14, 7, 2, 8, 1
$0 \to 1$	14, **4**, 7, 2, 8, 1
$1 \to 4$	14, 8, 7, 2, **4**, 1



El 4 desciende por los índices $0 \to 1 \to 4$. Los otros subárboles conservan su orden.

Flotar

Método de `HeapMaximo`; i es un índice activo y $n =$ tamano. Se acaba de añadir una hoja o aumentar la clave de un nodo de un max-heap.

Método Flotar(i)

$A \leftarrow$ datos
Mientras $i > 0$ y
  $A[\text{Padre}(i)].\text{dato} < A[i].\text{dato}$:
  $p \leftarrow \text{Padre}(i)$
  Intercambiar(i, p)
  $i \leftarrow p$

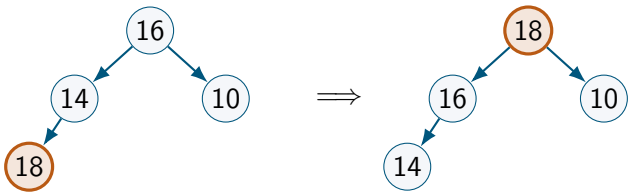
```
def flotar(self, i):
    A = self.datos
    while (i > 0 and
           A[padre(i)].dato
           < A[i].dato):
        p = padre(i)
        self.intercambiar(i, p)
        i = p
```

Costo: si el nodo debe ascender, $O(\log_2(n+1))$ en el peor caso. Si está en la raíz o ya cumple el orden con su padre, $\Theta(1)$.

Seguimiento de flotar

Partimos de $[16, 14, 10]$, con una casilla libre ($n = 3$, $m = 4$). Añadimos la hoja 18 y aplicamos `flotar(3)`.

Paso	Claves en el arreglo
Inicial	16, 14, 10, **18**
$3 \to 1$	16, **18**, 10, 14
$1 \to 0$	**18**, 16, 10, 14



El 18 asciende por $3 \to 1 \to 0$. Cada intercambio actualiza los índices de ambos nodos.

Construir un heap

Las hojas ya son max-heaps de un elemento. Aplicamos hundir a los nodos internos desde el último hasta la raíz.

Método Construir()

$n \leftarrow$ tamano
Para i desde $\lfloor n/2 \rfloor - 1$ hasta 0, decreciendo:
  Hundir(i)

```
def construir(self):
    n = self.tamano
    for i in range(n // 2 - 1, -1, -1):
        self.hundir(i)
```

Hojas: índices $\lfloor n/2 \rfloor, \ldots, n-1$.
Último nodo interno: $\lfloor n/2 \rfloor - 1$.

El tamaño activo lo fijamos al crear `HeapMaximo`. Con cero o un elemento, el ciclo no ejecuta ninguna iteración.

Seguimiento

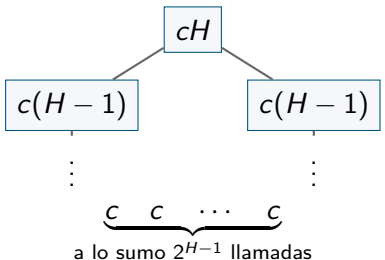
```
A = [NodoHeap(k) for k in [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]]
H = HeapMaximo(A, len(A))
H.construir()
```

Índice procesado	Claves después de la llamada
Entrada	4, 1, 3, 2, 16, 9, 10, 14, 8, 7
4	4, 1, 3, 2, 16, 9, 10, 14, 8, 7
3	4, 1, 3, 14, 16, 9, 10, 2, 8, 7
2	4, 1, 10, 14, 16, 9, 3, 2, 8, 7
1	4, 16, 10, 14, 7, 9, 3, 2, 8, 1
0	16, 14, 10, 8, 7, 9, 3, 2, 4, 1

Cada hijo ya encabeza un max-heap cuando se aplica hundir a su padre.

Costo de construir

Sea H la altura del heap, con $n \geq 1$. Cada nodo interno corresponde a una llamada iniciada por el ciclo de `construir`. Su etiqueta acota el costo **completo** de esa llamada a `hundir`.



Profundidad i, con $0 \leq i < H$

Llamadas	$\leq 2^i$
Altura restante	$\leq H - i$
Costo por llamada	$\leq c(H-i)$
Costo conjunto	$\leq c\,2^i(H-i)$

Las hojas no generan llamadas desde el ciclo.

En un nodo interno de altura $q \geq 1$, $q + 1 \leq 2q$: el costo $O(q+1)$ queda acotado por cq, para alguna constante $c > 0$.

Costo total

Partimos de $T(n) \leq d + c\sum_{i=0}^{H-1} 2^i(H-i)$, con $n \geq 1$. $d > 0$ acota el trabajo fuera del ciclo.

Cambio de índice y factorización

Tomamos $j = H - i$, de modo que $i = H - j$.

$$i = 0 \Rightarrow j = H, \qquad i = H-1 \Rightarrow j = 1.$$

Los mismos términos se recorren en orden inverso.

$$2^i(H-i) = 2^{H-j}j = \frac{2^H}{2^j}j.$$

Como 2^H no depende de j, sale de la suma:

$$\sum_{i=0}^{H-1} 2^i(H-i) = \sum_{j=1}^{H} 2^{H-j}j$$

$$= 2^H \sum_{j=1}^{H} \frac{j}{2^j}.$$

Cota superior

$$T(n) \leq d + c\,2^H \sum_{j=1}^{H} \frac{j}{2^j}$$

$$\leq d + c\,2^H \sum_{j=1}^{\infty} \frac{j}{2^j}$$

$$= d + 2c\,2^H$$

$$\leq d + 2cn$$

Sumandos positivos; $\sum_{j=1}^{\infty} j/2^j = 2$. Además, $2^H \leq n$.

Cota inferior: las $\lfloor n/2 \rfloor$ iteraciones dan $T(n) \in \Omega(n)$. Por ambas cotas, $\boxed{T(n) \in \Theta(n).}$

Cola de prioridad

Definición: cola de prioridad

Mantiene elementos con una clave asociada. En una cola de prioridad de máximos, se consulta o retira un elemento cuya clave es máxima.

Cola FIFO

Se atiende según el orden de llegada.

$$(\text{trabajo A}, \text{trabajo B}, \text{trabajo C}).$$

Primero se retira el trabajo A.

Cola de prioridad

Las claves determinan qué elemento se atiende.

$$A : 4, \qquad B : 9, \qquad C : 6.$$

Primero se retira el trabajo B.

Un **max-heap** implementa este comportamiento. Una cola de mínimos retira la menor clave; por ejemplo, el evento con menor tiempo programado.

Operaciones

Cola vacía con capacidad 20: `Q = HeapMaximo([None] * 20, 0)`.

Método	Efecto y resultado
`maximo()`	Consulta y devuelve un elemento máximo.
`extraer_maximo()`	Retira y devuelve un elemento máximo.
`aumentar_clave (elemento, k)`	Cambia la clave de un elemento del heap a k, con $k \geq$ elemento.dato. Devuelve `None`.
`insertar(elemento)`	Agrega un objeto nuevo con su clave establecida. Devuelve `None`.

`maximo()` y `extraer_maximo()` producen `IndexError` si está vacío. Insertar requiere espacio libre y un elemento que todavía no pertenece al heap.

Máximo

Método de `HeapMaximo`; devuelve una referencia al objeto, no solo su clave.

Método Maximo()

Si self.tamano $= 0$:
    Error: heap vacío.
Retornar self.datos[0]

```
def maximo(self):
    if self.tamano == 0:
        raise IndexError("heap vacio")
    return self.datos[0]
```

La propiedad de max-heap garantiza que ningún descendiente tiene una clave mayor que la raíz.

Costo: $\Theta(1)$. Consultar no modifica el heap.

Extraer máximo

El último elemento ocupa la raíz; después se restaura el orden hacia abajo.

Método ExtraerMaximo()

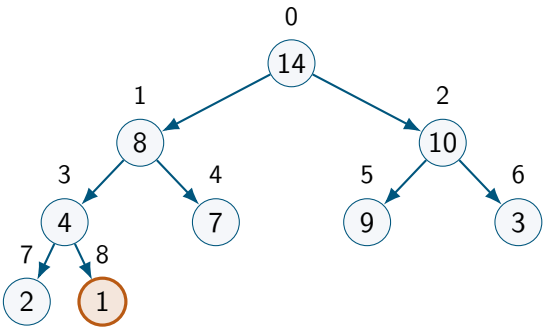
retirado ← Maximo()
$u \leftarrow$ self.tamano $- 1$
Si $u > 0$:
    self.datos[0] ← self.datos[u]
    self.datos[0].indice ← 0
self.datos[u] ← None
self.tamano ← u
retirado.indice ← None
Si $u > 0$: Hundir(0).
Retornar retirado.

```
def extraer_maximo(self):
    retirado = self.maximo()
    u = self.tamano - 1
    if u > 0:
        self.datos[0] = self.datos[u]
        self.datos[0].indice = 0
    self.datos[u] = None
    self.tamano = u
    retirado.indice = None
    if u > 0:
        self.hundir(0)
    return retirado
```

Con un solo elemento, queda tamaño 0 y no se llama a `hundir`.

Seguimiento

Retiramos el máximo 16 del heap que venimos utilizando.



El objeto con clave 1 ocupa primero la raíz y desciende por los índices:

$$0 \longrightarrow 1 \longrightarrow 3 \longrightarrow 8.$$

En cada paso se intercambia con el hijo de mayor clave.

El tamaño activo pasa de 10 a 9; la capacidad no cambia.

Costo: trabajo constante y una llamada a `hundir(0)`: $O(\log_2(n+1))$ en el peor caso, con n elementos antes de extraer.

Aumentar clave

Precondición: `nodo` pertenece al heap; su índice está actualizado.

Método AumentarClave(nodo, k)

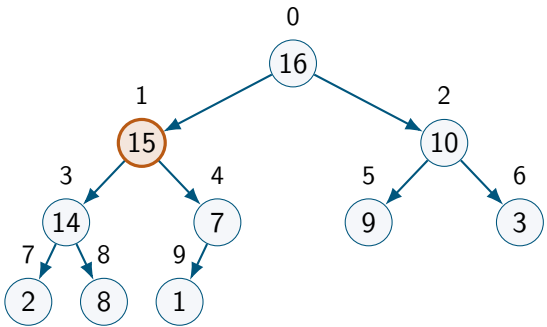
Si $k <$ nodo.dato:
    Error: la clave disminuiría.
nodo.dato $\leftarrow k$
Flotar(nodo.indice)

```
def aumentar_clave(self, nodo, k):
    if k < nodo.dato:
        raise ValueError("clave menor")
    nodo.dato = k
    self.flotar(nodo.indice)
```

Después de cambiar la clave, `flotar` restablece el orden ascendiendo.

Seguimiento

Aumentamos de 4 a 15 la clave del objeto ubicado inicialmente en el índice 8.



Índice	Decisión
8	$15 > 8$: intercambiar.
3	$15 > 14$: intercambiar.
1	$15 \leq 16$: terminar.

El mismo objeto asciende:

$$8 \longrightarrow 3 \longrightarrow 1.$$

Su atributo `indice` cambia con cada intercambio.

Costo: un solo camino hacia la raíz: $O(\log_2(n+1))$.

Insertar

Recibe un nodo nuevo con su clave establecida; debe haber una casilla libre.

Método Insertar(elemento)

Si self.tamano $=$ |self.datos|:
    Error: heap lleno.
$i \leftarrow$ self.tamano
self.datos[i] $\leftarrow$ elemento
elemento.indice $\leftarrow i$
self.tamano $\leftarrow$ self.tamano $+ 1$
Flotar(i)

```
def insertar(self, elemento):
    if self.tamano == len(self.datos):
        raise OverflowError(
            "heap lleno")
    i = self.tamano
    self.datos[i] = elemento
    elemento.indice = i
    self.tamano += 1
    self.flotar(i)
```

La nueva hoja conserva la forma completa; `flotar` restaura el orden.

Costos

Operación	Trabajo principal	Peor caso
Consultar máximo	Acceder a la raíz	$\Theta(1)$
Extraer máximo	Hundir desde la raíz	$O(\log_2(n+1))$
Aumentar clave	Flotar desde el nodo	$O(\log_2(n+1))$
Insertar	Añadir hoja y flotar	$O(\log_2(n+1))$